

AI Engineering Assessment

Build a RAG + Agentic Employee Assistant

Full Stack + Generative AI Fresher Assessment

Objective

Build a simple Employee AI Assistant that answers questions from company documents using Retrieval-Augmented Generation (RAG) and performs basic actions using an Agent with tools.

This assignment is designed to evaluate your understanding of the fundamental concepts behind LLM applications, RAG pipelines, vector databases, embeddings, prompt engineering, tool calling, backend APIs, and frontend integration.

Recommended Duration: 3–5 Hours

Problem Statement

You are given a small set of company documents (PDFs or text files) containing information such as:

- Leave Policy
- Work From Home Policy
- Travel Policy
- Employee Benefits
- IT & Security Policy
- Company FAQs

Build an AI assistant that can:

- Answer employee questions using the provided documents.
- Mention the source document(s) used to generate the answer.
- Respond with "I couldn't find this information in the provided documents." when the answer is unavailable.
- Perform simple employee-related actions using tools (agentic workflow).

Part 1 — Implement a Basic RAG Pipeline

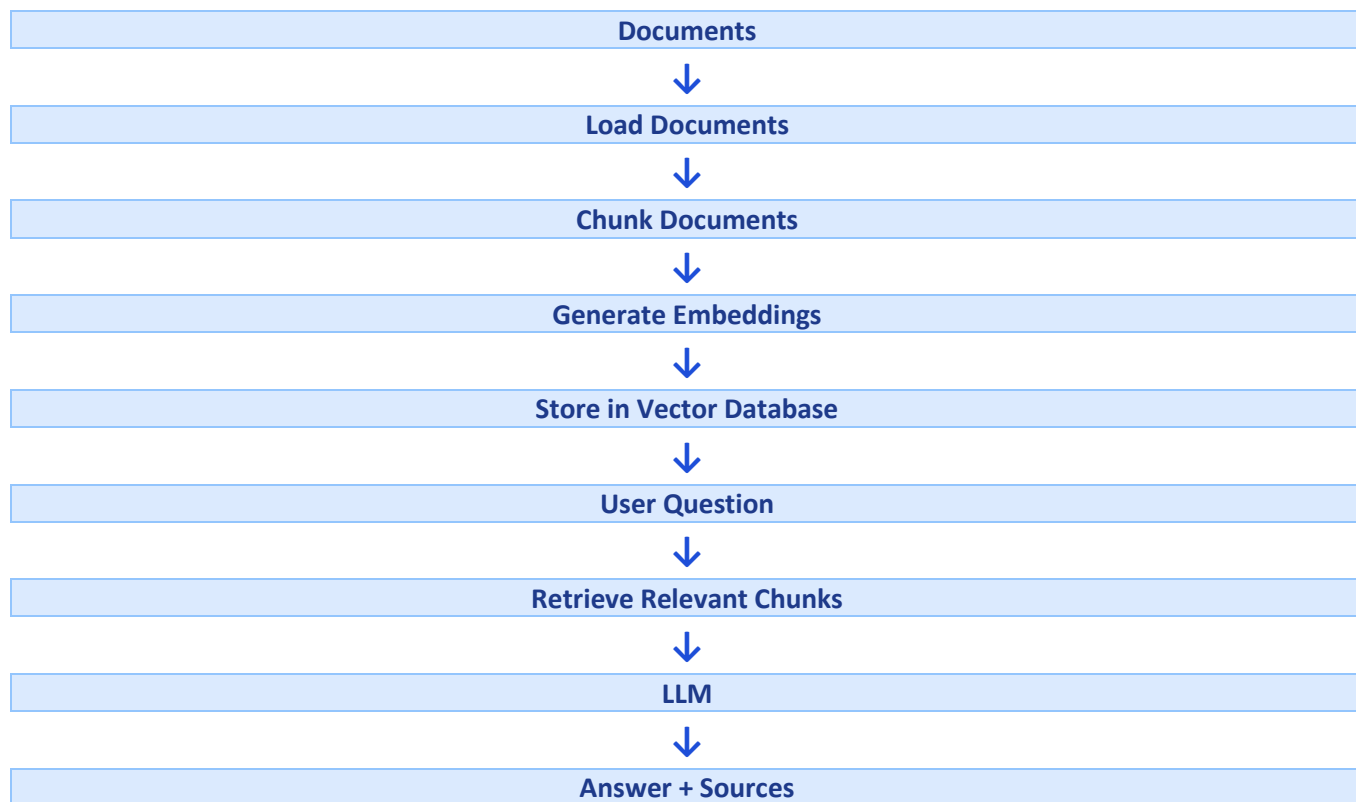
Implement a complete Retrieval-Augmented Generation pipeline.

Your RAG pipeline should include:

- Document loading
- Document chunking
- Embedding generation
- Storing embeddings in a vector database
- Retrieving relevant chunks

- Passing retrieved context to an LLM
- Returning an answer with document sources

Expected Flow



Example

Question: *What is the work from home policy?*

Expected Response: Employees can work from home up to 2 days a week with manager approval.

Source: *work_from_home_policy.pdf*

Hallucination Handling

If the information is not available:

Question: *Does the company provide pet insurance?*

Expected Response: I couldn't find information about pet insurance in the provided company documents.

Do not generate unsupported answers.

Part 2 — Build a Simple Agent

Extend your RAG application into a basic AI Agent that can decide when to use tools. Implement the following tools.

Tool 1 — Company Knowledge Search

Search the vector database.

Function

```
search_company_documents(query)
```

Use this tool whenever the user asks policy or FAQ-related questions.

Tool 2 — Employee Information

Create a mock employee database. Example:

```
{
  "EMP001": {
    "name": "Rahul",
    "department": "Engineering",
    "leave_balance": 12
  },
  "EMP002": {
    "name": "Priya",
    "department": "HR",
    "leave_balance": 8
  }
}
```

Function

```
get_employee_info(employee_id)
```

Tool 3 — Apply Leave

Create a mock leave application function.

Function

```
apply_leave(employee_id, start_date, end_date, reason)
```

Return a success or failure response. Example:

```
{
  "status": "success",
  "message": "Leave application submitted successfully."
}
```

Part 3 — Agentic Workflow

Your agent should choose the correct tool(s) based on user intent.

Scenario 1 — RAG Only

Question: *What is the leave policy?*

Flow:

- Search company documents.
- Retrieve relevant chunks.
- Generate answer.

Scenario 2 — Tool Only

Question: *How many leaves do I have remaining?*

Flow:

- Fetch employee information.
- Return leave balance.

Scenario 3 — Multiple Tools

Question: *What is the leave policy and how many leaves do I have remaining?*

Flow:

- Search company documents.
- Fetch employee information.
- Combine both responses.

Scenario 4 — Action Request

Question: *Apply leave for EMP001 from 20 September to 22 September because I'm travelling.*

Flow:

- Fetch employee information.
- Check leave balance.
- Submit leave request.
- Return confirmation.

Part 4 — Conversational Context

Maintain basic conversation history.

Example

User: How many leaves do I have?

Assistant: You have 12 days remaining.

User: Can I take 3 days next month?

The assistant should understand that the follow-up refers to leave.

Part 5 — Frontend + Backend

Implement a complete application.

Backend (Mandatory)

Create backend APIs using a framework of your choice. Example:

```
POST /chat
```

Request:

```
{
  "employee_id": "EMP001",
  "message": "How many leaves do I have?"
}
```

Response:

```
{
  "answer": "You have 12 days remaining.",
  "sources": [],
  "tools_used": ["get_employee_info"]
}
```

Backend should include:

- API endpoints.
- RAG pipeline.
- Agent implementation.
- Tool calling.
- Error handling.
- Configuration using environment variables.

Frontend (Mandatory)

Build a simple chat interface where a user can:

- Ask questions.
- View assistant responses.
- View document sources.
- See tool usage (optional but recommended).

You may use React, Next.js, or any frontend framework of your choice. UI does not need to be highly polished, but it should be functional.

Technical Requirements

You may use any suitable technology stack. Examples include:

Frontend

- React
- Next.js
- HTML/CSS/JavaScript

Backend

- Python
- FastAPI
- Flask

AI Frameworks (Optional)

- LangChain
- LlamaIndex
- Plain Python implementation

Vector Database

- FAISS
- ChromaDB
- Qdrant
- Pinecone (optional)

LLM / Embeddings

- OpenAI
- Gemini
- Ollama
- Hugging Face
- Any equivalent model

Submission Deliverables

1. Source Code

- GitHub Repository (preferred) or ZIP file.

2. Working Frontend

A functional UI for interacting with the assistant.

3. Working Backend

API implementation with RAG and Agent workflow.

4. README

Include:

- Setup instructions.
- Project architecture.
- Technologies used.
- Chunking strategy.
- Embedding model used.
- Vector database used.
- Retrieval approach.
- Agent/tool implementation.
- Assumptions.
- Limitations.

5. Architecture Diagram

Include a simple diagram showing:

- Document ingestion.
- Embedding pipeline.
- Vector database.
- LLM.
- Agent.
- Tools.
- Frontend ↔ Backend interaction.

6. Sample Queries

Include at least 5 sample questions demonstrating:

- RAG.
- No-answer case.
- Tool usage.
- Multi-tool workflow.
- Leave application.

Sample Test Cases

User Query	Expected Capability
What is the work from home policy?	RAG retrieval
How many annual leaves are allowed?	RAG retrieval
Does the company provide pet insurance?	Hallucination prevention
How many leaves does EMP001 have?	Tool calling
What is the leave policy and how many leaves does EMP001 have?	Multi-tool reasoning
Apply leave for EMP001 from 20 Sept to 22 Sept.	Agent action
How many leaves will I have after applying?	Conversational context

Important Instructions

- You may use AI coding assistants (ChatGPT, GitHub Copilot, Cursor, Claude, Gemini, etc.) while completing this assignment.
- However, you are expected to fully understand every part of your implementation.
During the technical interview, you should be able to explain:
 - How your RAG pipeline works.
 - Why you chose your chunking strategy.
 - How embeddings and retrieval work.
 - How the agent decides which tool to call.
 - How the frontend communicates with the backend.
 - Any design decisions, assumptions, and limitations.
- We may ask you to modify or extend a part of your implementation during the interview.

Note: We are evaluating your understanding of concepts, implementation approach, and problem-solving ability—not just whether the application works.

Bonus (Optional)

Implement any of the following for additional credit:

- Streaming responses.
- Conversation memory.
- Metadata filtering.
- Reranking.
- Retrieval confidence threshold.
- Docker setup.
- Unit tests.
- Logging/Observability.

- Prompt injection protection.
- Basic authorization for employee actions.

Submission Deadline

Please submit your completed assignment by the deadline communicated in the email.

All the best!